CySec Don

FOR EDUCATIONAL AND DEFENSIVE USE ONLY

COMPREHENSIVE PROFESSIONAL HANDBOOK + HANDS-ON LAB

SQL and SQL Injection

A Complete Guide to Structured Query Language and SQL Injection defense. Includes a custom-built training lab with Docker deployment, eight progressive challenges, and full source code for build-from-source installation. Covers database fundamentals through advanced prevention and mitigation strategies.

Defensive Security & Ethical Testing Focus

With CySec Don SQLi Training Lab

Table of Contents

Part I: Foundations of SQL

Chapter 1: Introduction to Databases

A database is an organized collection of structured information, or data, typically stored electronically in a computer system. A database is usually controlled by a Database Management System (DBMS), which provides an interface between the data and the applications or end users that need to interact with it. Think of a database as a highly organized filing cabinet where every piece of information has a specific place, is easy to find, and can be cross-referenced with other information at a moment's notice. The concept of databases dates back to the 1960s, but the relational model, which underpins most modern databases, was introduced by Edgar F. Codd at IBM in 1970 and revolutionized how data is stored, queried, and managed.

Databases are the backbone of virtually every modern application. When you check your bank balance, browse an online store, book a flight, or scroll through social media, you are interacting with one or more databases behind the scenes. Without databases, the digital world as we know it would simply not exist. They enable organizations to manage vast quantities of data efficiently, ensure data integrity and consistency, support concurrent access by multiple users, and provide security mechanisms to protect sensitive information from unauthorized access.

Relational vs. Non-Relational Databases

Relational databases (RDBMS) organize data into tables with rows and columns, linked by defined relationships. They follow the relational model introduced by Codd and use Structured Query Language (SQL) for data manipulation. Examples include MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server. Relational databases enforce a strict schema, meaning the structure of the data must be defined before any data can be inserted. This rigidity provides strong data consistency and integrity guarantees, making relational databases ideal for applications where the structure of data is well-defined and unlikely to change frequently.

Non-relational databases (NoSQL) emerged in the late 2000s to address the limitations of relational databases at massive scale. They offer flexible schemas, horizontal scalability, and specialized data models optimized for specific use cases. Document databases like MongoDB store data in JSON-like documents; key-value stores like Redis excel at caching and session management; column-family stores like Apache Cassandra handle massive write-heavy workloads; and graph databases like Neo4j model complex relationships between entities efficiently.

Feature	Relational (SQL)	Non-Relational (NoSQL)
Schema	Strict, predefined	Flexible, dynamic
Query Language	SQL	Varies (API, query language)

Scaling	Vertical (scale-up)	Horizontal (scale-out)
Data Model	Tables, rows, columns	Documents, key-value, graph
ACID Compliance	Strong	Eventual (varies)
Best For	Structured data, transactions	Unstructured data, high scale
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra

Real-World Use Cases

In the financial sector, relational databases power banking systems that must process millions of transactions daily with absolute accuracy. The ACID (Atomicity, Consistency, Isolation, Durability) properties of relational databases provide the guarantees that financial institutions require. In e-commerce, databases manage product catalogs, customer accounts, order processing, inventory tracking, and recommendation engines. Social media platforms use massive distributed databases to store user profiles, posts, and the complex web of interactions that occur every second. Healthcare systems rely on databases to maintain electronic health records and ensure that sensitive medical information is accessible only to authorized personnel.

Chapter Summary

- A database is an organized collection of data managed by a DBMS.
- Relational databases use tables with fixed schemas; NoSQL databases offer flexible schemas.
- SQL is the standard language for relational database interaction.
- Databases power banking, e-commerce, social media, and healthcare systems.
- Choosing between SQL and NoSQL depends on data structure, scale, and consistency needs.

Chapter 2: Introduction to SQL

Structured Query Language (SQL) is the standard programming language used to manage, manipulate, and query data held in relational databases. SQL was first developed at IBM in the early 1970s by Donald D. Chamberlin and Raymond F. Boyce, originally called SEQUEL (Structured English QUEry Language). The language was designed to be intuitive, using English-like syntax that allows users to describe what data they want without specifying exactly how to retrieve it. This declarative nature is one of SQL's most powerful features: you tell the database what you want, and the database's query optimizer figures out the most efficient way to get it.

History and Evolution

SQL has undergone several major revisions since its inception. SQL-86 was the first standardized version, followed by SQL-89, SQL-92 (which introduced JOIN operations), SQL:1999 (which added

recursive queries and triggers), SQL:2003 (which introduced XML support and window functions), SQL:2008, SQL:2011 (temporal data), and SQL:2016/2019/2023 (JSON support, property graph queries). Despite these evolution cycles, the core of SQL has remained remarkably stable. A basic SELECT statement written in 1992 will still work perfectly in the latest version of PostgreSQL or MySQL.

Common Database Systems

MySQL is the world's most popular open-source relational database, powering millions of web applications. **PostgreSQL** is often described as the most advanced open-source database, supporting advanced data types, full-text search, and a powerful extension system. **Microsoft SQL Server (MSSQL)** is a commercial relational database widely used in enterprise environments, particularly within the Windows ecosystem. **Oracle Database** is the dominant commercial database in large enterprise environments, particularly in banking, telecommunications, and government sectors.

Chapter Summary

- SQL is the standard language for managing relational databases, developed at IBM in the 1970s.
- SQL is declarative: you describe what data you want, not how to get it.
- Major SQL standards: SQL-92 (foundational), SQL:1999 (triggers), SQL:2003 (window functions).
- MySQL: most popular open-source RDBMS. PostgreSQL: most feature-rich.
- MSSQL: dominant in Windows/enterprise environments. Oracle: enterprise-grade, battle-tested.

Chapter 3: SQL Syntax and Core Concepts

Understanding SQL requires a solid grasp of its foundational concepts: tables, rows, columns, keys, and constraints. A table represents a single entity type, such as "users," "products," or "orders." Each column represents a specific property, and each row represents a single instance. Data types define what kind of data each column can hold: INT, VARCHAR(n), TEXT, DATE, TIMESTAMP, BOOLEAN, DECIMAL(p,s), and BLOB.

```
CREATE TABLE users ( user_id INT PRIMARY KEY AUTO_INCREMENT, username
VARCHAR(50) NOT NULL UNIQUE, email VARCHAR(255) NOT NULL UNIQUE, password_hash
VARCHAR(255) NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
is_active BOOLEAN DEFAULT TRUE );
```

Primary Keys and Foreign Keys

A primary key uniquely identifies each row in a table. A foreign key establishes and enforces relationships between tables, ensuring referential integrity. Constraints (NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY) are rules enforced by the database that maintain data integrity and consistency.

Constraint	Purpose	Example
PRIMARY KEY	Unique row identification	user_id INT PRIMARY KEY
FOREIGN KEY	References another table	user_id INT REFERENCES users(id)
NOT NULL	Column cannot be empty	email VARCHAR(255) NOT NULL
UNIQUE	No duplicate values	username VARCHAR(50) UNIQUE
CHECK	Value must satisfy condition	age INT CHECK (age >= 0)
DEFAULT	Default value if none given	status VARCHAR(20) DEFAULT 'active'

Chapter Summary

- Tables store data in rows and columns, with each table representing an entity type.
- Data types (INT, VARCHAR, DATE, etc.) define what each column can hold.
- Primary keys uniquely identify rows; foreign keys link tables together.
- Constraints enforce data integrity and prevent invalid data.

Part II: Types of SQL Commands

SQL commands are organized into several sub-languages: DDL, DML, DCL, and TCL. Each serves a distinct purpose in the management and manipulation of database systems.

Chapter 4: DDL - Data Definition Language

Data Definition Language (DDL) defines, modifies, and destroys the structural elements of a database. The three primary DDL commands are CREATE, ALTER, and DROP.

```
CREATE TABLE employees ( emp_id INT PRIMARY KEY AUTO_INCREMENT, first_name
VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL, email VARCHAR(255) NOT
NULL UNIQUE, department VARCHAR(50), salary DECIMAL(10,2) CHECK (salary > 0) );
```

ALTER modifies existing structures. DROP permanently removes database objects and all their data. DROP is irreversible and one of the most dangerous SQL commands, making it a favorite target in SQL injection attacks.

Chapter Summary

- DDL defines database structure: CREATE, ALTER, DROP.
- DROP is irreversible and dangerous, especially via SQL injection.
- Good schema design is the foundation of database security.

Chapter 5: DML - Data Manipulation Language

DML interacts with actual data: SELECT (retrieve), INSERT (add), UPDATE (modify), DELETE (remove). These are the most frequently used SQL commands in application development and the primary targets of SQL injection attacks.

```
-- Basic SELECT SELECT first_name, last_name FROM employees WHERE department =
'Engineering' AND salary > 80000 ORDER BY salary DESC LIMIT 10;
```

UPDATE and DELETE without WHERE clauses modify/delete ALL rows, making them extremely dangerous. In SQL injection, attackers manipulate these queries to change passwords, elevate privileges, or destroy data across entire tables.

Chapter Summary

- DML manipulates data: SELECT, INSERT, UPDATE, DELETE.
- UPDATE and DELETE without WHERE clauses affect ALL rows.
- SQL injection frequently targets DML statements.
- Always use WHERE clauses with UPDATE and DELETE.

Chapter 6: DCL - Data Control Language

DCL manages database access: GRANT gives permissions, REVOKE removes them. The principle of least privilege dictates that applications should only have the minimum permissions necessary. Even if an attacker achieves SQL injection, proper DCL restrictions limit the damage.

> **Security Insight:** If the application user only has SELECT permissions, an injected DROP TABLE statement will fail with a permission denied error. This is the principle of least privilege in action.

Chapter Summary

- DCL manages access: GRANT gives, REVOKE removes permissions.
- Principle of least privilege: grant only minimum necessary permissions.
- Proper DCL configuration is critical defense-in-depth against SQL injection.

Chapter 7: TCL - Transaction Control Language

TCL manages transactions: COMMIT makes changes permanent, ROLLBACK undoes them, SAVEPOINT sets markers for partial rollback. Transactions ensure that groups of operations succeed or fail as a single atomic unit, maintaining data integrity.

```
BEGIN TRANSACTION; UPDATE accounts SET balance = balance - 500 WHERE account_id
= 1; UPDATE accounts SET balance = balance + 500 WHERE account_id = 2; COMMIT;
```

Chapter Summary

- Transactions group operations into atomic units.
- COMMIT makes changes permanent; ROLLBACK undoes all changes.
- ACID properties: Atomicity, Consistency, Isolation, Durability.
- Security researchers should use transactions to roll back test changes.

Part III: Advanced SQL

This part explores advanced SQL features essential for building efficient, secure applications and understanding attack surfaces.

Joins

Joins combine rows from multiple tables based on related columns. INNER JOIN returns only matching rows; LEFT JOIN returns all rows from the left table plus matches; RIGHT JOIN returns all rows from the right table; FULL OUTER JOIN returns all rows from both tables. Attackers can manipulate JOIN clauses to extract data from unauthorized tables.

```
SELECT e.first_name, d.department_name FROM employees e INNER JOIN departments
d ON e.department_id = d.department_id;
```

Subqueries

Subqueries are queries nested within other queries. They can appear in SELECT, FROM, WHERE, and HAVING clauses. Correlated subqueries reference columns from the outer query, while non-correlated subqueries are self-contained.

Indexing and Optimization

Indexes improve data retrieval speed at the cost of additional storage and slower writes. From a security perspective, indexes can make data extraction attacks faster, as indexed columns produce quicker responses in blind injection attacks.

Stored Procedures

Stored procedures are pre-compiled SQL statements stored in the database. They improve performance and can enhance security through abstraction, but can introduce vulnerabilities if they construct dynamic SQL from user input without proper parameterization.

Views and Triggers

Views are virtual tables that restrict data access. Triggers automatically execute in response to database events. Triggers can detect suspicious activity for auditing purposes, but can also be exploited for persistent access if an attacker can create or modify them.

Chapter Summary

- Joins combine data from multiple tables using various types.
- Subqueries allow queries within queries for complex retrieval.
- Indexes improve query speed but can aid attacker data extraction.
- Stored procedures encapsulate logic but can be vulnerable with dynamic SQL.
- Views restrict data access; triggers automate responses to data changes.

Part IV: Introduction to SQL Injection (SQLi)

SQL injection (SQLi) is one of the oldest, most prevalent, and most dangerous web application security vulnerabilities. It occurs when an attacker can insert malicious SQL code into a query that the application sends to its database. SQL injection has been a consistent top threat in the OWASP Top 10 since its inception.

What is SQL Injection?

SQL injection exploits the failure to separate code (the SQL query structure) from data (user-supplied input). Consider a login query:

```
SELECT * FROM users WHERE username = 'john' AND password = 'secret';
```

If an attacker enters admin' -- as the username, the query becomes:

```
SELECT * FROM users WHERE username = 'admin' --' AND password = 'anything';
```

The double dash comments out the password check, and the attacker is logged in as admin without knowing the password.

Types of SQL Injection

In-Band SQL Injection uses the same channel to inject and retrieve results. Error-based SQLi relies on error messages to gather information. Union-based SQLi uses the UNION operator to combine injected query results with original results.

Blind SQL Injection occurs when results are not directly visible. Boolean-based blind SQLi infers data from true/false response differences. Time-based blind SQLi uses database timing functions (SLEEP, WAITFOR) to infer results through response delays.

Out-of-Band SQL Injection uses secondary channels (DNS, HTTP) to exfiltrate data when direct extraction is not possible. **Second-Order SQL Injection** stores payloads in the database that are triggered by a separate action later.

Chapter Summary

- SQL injection exploits mixing SQL code with user-supplied data.
- In-band: Error-based (error messages) and Union-based (combined results).
- Blind: Boolean-based (true/false responses) and Time-based (timing delays).
- Out-of-band: Uses DNS/HTTP secondary channels.
- Second-order: Payload stored in DB, triggered later in different context.

Part V: How SQL Injection Works (Defensive Analysis)

Understanding SQL injection mechanics is essential for building effective defenses. The fundamental flaw is mixing code and data when constructing database queries.

Input Handling Flaws

```
// VULNERABLE: Direct string concatenation $query = "SELECT * FROM users WHERE
username = '" . $_POST['username'] . "' AND password = '" . $_POST['password']
. "'";
```

The fundamental defense is parameterized queries, which send SQL structure and data separately, making injection impossible.

Authentication Bypass

```
-- Injecting admin' OR '1'='1' -- as username: SELECT * FROM users WHERE
username = 'admin' OR '1'='1' --'
```

Defense requires parameterized queries plus rate limiting, account lockout, multi-factor authentication, and logging.

Common Vulnerable Patterns

Language	Vulnerable Pattern	Secure Alternative
PHP	"SELECT * FROM t WHERE id=".$_GET["id"]	PDO prepared statements
Python	"SELECT * FROM t WHERE id=%s" % id	parameterized cursor.execute()
Java	Statement.executeQuery(sql+input)	PreparedStatement with ?
Node.js	"SELECT * FROM t WHERE id="+req.params.id	mysql2 with ? placeholders
C#/.NET	"SELECT * FROM t WHERE id="+Request["id"]	SqlParameter objects
Ruby	"SELECT * FROM t WHERE id=#{params[:id]}"	ActiveRecord sanitization

Chapter Summary

- SQL injection is caused by mixing code with user data.
- String concatenation and dynamic query building are root causes.
- Authentication bypass uses tautologies or comment injection.
- Every language has both vulnerable and secure patterns.

Part VI: Detection and Testing

Detecting SQL injection before attackers find it is critical. All testing should only be performed on systems you own or have explicit authorization to test.

Identifying Vulnerable Parameters

Test all user-controlled inputs: URL parameters, form fields, HTTP headers, cookies, and API endpoints. Initial test payloads include single quote ('), double dash (--), semicolon (;), and AND 1=1 / AND 1=2 conditions. Different responses indicate potential vulnerability.

Logging and Monitoring

Log all database queries with parameters, monitor for SQL injection signatures in WAFs, track failed login attempts, and alert on anomalous query patterns. Centralized log management (ELK stack, Splunk)

provides comprehensive attack visibility.

Safe Use of Tools

SQLMap, OWASP ZAP, and Burp Suite assist in detection. Always test only authorized systems, stay within scope, and never exploit beyond what is necessary to confirm a vulnerability. Tools are powerful but no substitute for understanding fundamentals.

Chapter Summary

- Test all user-controlled inputs systematically.
- Response types: error-based, boolean-based, time-based, out-of-band.
- Log all queries, monitor for injection signatures, set up anomaly alerts.
- Tools assist testing but require skilled operators.

Part VII: Prevention and Defense

Preventing SQL injection requires a comprehensive defense-in-depth strategy.

Parameterized Queries

```
cursor.execute( "SELECT * FROM users WHERE username = ? AND password = ?",
(username, password) )
```

Parameterized queries separate SQL structure from data, making injection fundamentally impossible. They are supported by every modern database driver and programming language.

ORM, Input Validation, Least Privilege, WAF

ORM frameworks auto-generate parameterized queries for standard operations. Input validation (server-side allowlisting) adds a defense-in-depth layer. The principle of least privilege limits blast radius. Web Application Firewalls provide pattern-based protection but can be bypassed.

Defense Layer	Technique	Effectiveness
Primary	Parameterized Queries	Near-Complete
Primary	ORM (used correctly)	High
Secondary	Input Validation	Moderate
Secondary	Least Privilege (DCL)	High

Tertiary	Web Application Firewall	Moderate
Monitoring	Logging and Alerting	Detective

Chapter Summary

- Parameterized queries are the gold standard defense.
- ORMs provide automatic protection when used correctly.
- Input validation, least privilege, and WAFs add defense-in-depth.
- Combine multiple layers for robust protection.

Part VIII: Hands-On Lab (CySec Don SQLi Training Lab)

This part provides a complete, professional hands-on lab environment that incorporates every concept covered in this textbook. The **CySec Don SQLi Training Lab** is a custom-built, intentionally vulnerable web application with eight progressive challenges, three security levels, a scoring system, and Docker-based deployment. It is specifically designed to accompany this textbook and provide practical experience with every SQL injection type discussed in Parts IV through VII.

Introducing the CySec Don SQLi Training Lab

The CySec Don SQLi Training Lab is a professional-grade, intentionally vulnerable Flask web application backed by a MariaDB database. It provides a safe, controlled environment for practicing SQL injection detection, analysis, and defense. The lab includes the following key features that directly correspond to concepts taught in this textbook:

- Eight progressive challenges covering all SQLi types (in-band, blind, out-of-band, second-order, privilege escalation, and data extraction)
- Three security levels (Low = vulnerable, Medium = partially protected, High = secure) demonstrating how defenses progressively mitigate attacks
- Real-time monitoring with login attempt logging and audit trails (corresponding to Part VI)
- A progress tracking system with scoring and completion badges
- phpMyAdmin integration for direct database query analysis
- One-command Docker deployment for instant lab setup
- Full source code availability for build-from-source installation

> **Companion Software:** The complete lab source code, Docker configuration, and build scripts are distributed alongside this textbook. All lab exercises in this chapter use the CySec Don

SQLi Training Lab as the target environment.

Lab Deployment Options

The CySec Lab can be deployed using three methods. Choose the one that best fits your environment and preferences.

Option A: Docker Compose (Recommended)

Docker Compose provides the fastest path to a running lab, launching the Flask application, MariaDB database, and phpMyAdmin as interconnected containers with a single command. This is the recommended approach for most users.

```
# Step 1: Navigate to the lab directory cd CySec_Lab # Step 2: Build and start all services docker compose up -d --build # Step 3: Wait for services to initialize (~30 seconds) docker compose ps # Step 4: Access the lab # Lab Application: http://localhost:5000 # phpMyAdmin: http://localhost:8080
```

The Docker Compose setup creates three services: a MariaDB 11.4 database container (with automatic schema initialization from init.sql), the Flask application container (which waits for the database to be healthy before starting), and a phpMyAdmin container for direct database access. All services communicate over an isolated Docker network, and database data persists across restarts via a Docker volume.

Option B: Docker Run (Standalone)

For more granular control, you can start the database and application containers individually using docker run commands:

```
# Start MariaDB docker run -d --name cysec-db \ -e MARIADB_ROOT_PASSWORD=root_password \ -e MARIADB_DATABASE=cysec_lab \ -e MARIADB_USER=cysec_lab \ -e MARIADB_PASSWORD=cysec_lab_pass \ -v $(pwd)/db/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro \ -p 3306:3306 mariadb:11.4 # Build and start the app docker build -t cysec-lab . docker run -d --name cysec-app --link cysec-db:db -p 5000:5000 cysec-lab
```

Option C: Build from Source

For maximum transparency and learning, you can build and run the lab directly from source code using Python and MariaDB. This approach is recommended for students who want to read and modify the vulnerable code to understand exactly how each vulnerability works:

```
# Step 1: Navigate to the lab directory cd CySec_Lab # Step 2: Run the build script bash scripts/build.sh # Step 3: Activate environment and start source venv/bin/activate export $(cat .env | xargs) python -m app.main # Step 4: Open http://localhost:5000
```

The build script automates prerequisite checking, virtual environment creation, dependency installation, database configuration, schema import, and environment variable setup. For manual setup instructions, refer to the README.md file included in the lab distribution.

Lab Architecture

The CySec Lab uses a three-tier architecture common in web applications: a Flask (Python) frontend handling HTTP requests and rendering templates, a MariaDB backend storing all application data, and an optional phpMyAdmin instance for direct database exploration. This architecture mirrors real-world applications, making the skills practiced in the lab directly transferable to professional security assessments.

Component	Technology	Port	Purpose
Web Application	Flask 3.0 (Python 3.12)	5000	Intentionally vulnerable lab interface
Database	MariaDB 11.4	3306	Sample data with users, products, cards, orders
DB Admin	phpMyAdmin 5.2	8080	Direct SQL query analysis tool

The database contains nine sample users, twelve cybersecurity-themed products, five credit card records (masked for realism), five orders, and audit tables for login attempts and change tracking. All passwords hash to the bcrypt value of "password" for lab convenience. The complete schema is defined in db/init.sql and is automatically initialized when the database container starts.

Challenge Walkthrough

Each challenge in the lab corresponds directly to concepts taught in this textbook. The following walkthrough connects each challenge to the relevant book chapters and provides guidance for completion.

Challenge 1: Authentication Bypass (100 pts)

Textbook Reference: Chapter 5 (DML), Part IV (What is SQL Injection), Part V (Authentication Bypass Logic). This challenge demonstrates the most classic SQL injection attack: bypassing a login form. Navigate to the Login page and observe that the application constructs the authentication query using direct string concatenation at the LOW security level. Try injecting a single quote to trigger an error, then use a classic tautology payload such as admin' OR '1'='1' -- to bypass authentication. When successful, you will be logged in as the admin user, and the challenge will be marked as complete.

Lab Exercise: After completing the challenge at LOW security, switch to MEDIUM and HIGH levels using the dashboard controls. Observe how the same payloads are blocked by escaping (MEDIUM) and parameterized queries (HIGH). This directly demonstrates the defense concepts from Part VII.

Challenge 2: Error-Based SQLi (150 pts)

Textbook Reference: Part IV (In-Band SQL Injection, Error-Based). Navigate to the Product Search page and enter a single quote (') in the search field. At the LOW security level, the database will return a detailed error message that reveals the query structure. Experiment with payloads that reference the information_schema tables to extract metadata. Any query containing "information_schema" that triggers a database response will complete the challenge.

Lab Exercise: Use the error messages to map the database schema. Then compare the error output at LOW level (detailed errors) with MEDIUM level (generic "An error occurred" message) to understand how error suppression reduces information leakage, as discussed in Part VI.

Challenge 3: Union-Based SQLi (200 pts)

Textbook Reference: Part IV (Union-Based SQL Injection). Navigate to the Union Search page (accessible from the dashboard). The original query returns three columns (product_id, name, price). First, confirm the column count using ORDER BY clauses, then craft a UNION SELECT statement with three columns to retrieve data from other tables. For example: ' UNION SELECT username, password_hash, email FROM users --. Any successful UNION injection that returns results will complete the challenge.

Lab Exercise: Progressively extract data: first from the users table, then from the credit_cards table (this also completes Challenge 7: Full Data Extraction). Use phpMyAdmin (port 8080) to verify the extracted data matches what is actually stored in the database.

Challenge 4: Boolean-Based Blind SQLi (250 pts)

Textbook Reference: Part IV (Boolean-Based Blind SQLi), Part VI (Understanding Application Responses). Navigate to the User Lookup page. Enter a user ID (1-9) to see normal results. Then inject a condition such as 1 AND 1=1 (should return the user) versus 1 AND 1=2 (should return nothing). Use substring-based extraction to determine individual characters of sensitive data, such as the admin password hash. Any injection containing both a logical operator (AND/OR) and a data extraction function (SUBSTR/SUBSTRING/SELECT) will complete the challenge.

Lab Exercise: Write a systematic extraction methodology: determine the length of the target string using LENGTH(), then extract each character using binary search with SUBSTRING(). Document how many queries it takes to extract a complete value and discuss how this demonstrates why blind SQLi is slower but still effective.

Challenge 5: Time-Based Blind SQLi (350 pts)

Textbook Reference: Part IV (Time-Based Blind SQLi). Navigate to the Ping Check page. This challenge simulates a scenario where the application returns identical responses regardless of query results (no boolean difference). Inject timing functions such as: 1 AND SLEEP(5) -- or 1 AND IF(1=1,SLEEP(3),0) --. If the response takes noticeably longer than normal, the condition evaluated to true. Any injection containing SLEEP or BENCHMARK will complete the challenge.

Lab Exercise: Use a stopwatch or browser developer tools to measure response times. Compare response times for true conditions (SLEEP executes) versus false conditions (SLEEP is skipped). Discuss how time-based injection is the slowest but most universally applicable blind technique.

Challenge 6: Second-Order SQLi (400 pts)

Textbook Reference: Part IV (Second-Order SQL Injection). This two-step challenge involves two pages. First, navigate to the Register page and create a new account with a SQL injection payload in the username field (e.g., test' OR '1'='1). Then navigate to the Profile Search page and search for the username you registered. The stored payload will be used unsafely in the profile search query, triggering the second-order injection. Any profile search with a username containing single or double quotes will complete the challenge.

Lab Exercise: Discuss why second-order injection is particularly dangerous in production systems: the initial input might pass through multiple layers of validation, but the stored payload is later used in a different context where those validations do not apply.

Challenge 7 and 8: Data Extraction and Privilege Escalation

Textbook Reference: Part IV (types of SQLi), Part V (Common Vulnerable Patterns), Part IX (Real-World Case Studies). Challenge 7 (500 pts) is automatically completed when you extract data from the credit_cards table using any injection technique. Challenge 8 (600 pts) involves the Admin Update Role page, where you can use injection to escalate a regular user to admin privileges. These challenges simulate the real-world impact described in Part IX: data theft and privilege escalation, the exact techniques used in the Heartland, Sony, and TalkTalk breaches.

Security Level Comparison

One of the most valuable learning features of the CySec Lab is the ability to switch between three security levels and observe how the same payloads are handled differently. This directly demonstrates the defense concepts from Part VII:

Security Level	Input Handling	Error Display	Challenge Outcome
LOW	Direct concatenation (no protection)	Full database errors exposed	All payloads succeed
MEDIUM	Basic escaping and validation	Generic error messages	Most payloads blocked
HIGH	Parameterized queries	No SQL errors possible	No injection possible

Switch between levels using the buttons on the dashboard. After completing a challenge at LOW level, immediately re-test your payloads at MEDIUM and HIGH to observe how each defense layer blocks the attack. This progressive comparison is the most effective way to internalize why each defense technique is important.

Database Exploration

Use phpMyAdmin (http://localhost:8080) with credentials cysec_lab / cysec_lab_pass to explore the database directly. This is invaluable for understanding the schema behind each challenge and verifying the results of your injections. Key tables to examine include:

- users: Contains admin accounts, salary data, and role assignments
- products: The target table for search and UNION-based challenges
- credit_cards: Contains masked card data (demonstrates data extraction risk)
- login_attempts: Audit trail recording all login attempts (Part VI monitoring)
- audit_log: Trigger-based change log for privilege escalation detection

Resetting the Lab

After experimenting with injections, you can reset the database to its original state by navigating to http://localhost:5000/reset-db. This re-executes the init.sql script, restoring all tables and sample data. For Docker installations, you can also reset by running docker compose down -v followed by docker compose up -d to recreate all containers and volumes from scratch.

Default Lab Credentials

Account	Username	Password	Role
Administrator	admin	password	admin
Regular User	alice	password	user

Pentest Account	pentest	password	admin
Guest	mallory	password	guest

Chapter Summary

- The CySec Don SQLi Training Lab is a professional, custom-built environment for hands-on practice.
- Deploy instantly with Docker Compose: docker compose up -d --build
- Eight challenges cover all SQLi types from this textbook (error-based, union-based, boolean blind, time blind, second-order, data extraction, privilege escalation).
- Three security levels (Low/Medium/High) demonstrate progressive defense implementation.
- phpMyAdmin integration enables direct database exploration and injection verification.
- Build from source for full code transparency and modification capability.
- Reset the lab anytime at http://localhost:5000/reset-db or with docker compose down -v.

Part IX: Real-World Defensive Case Studies

Studying real-world SQL injection incidents provides invaluable lessons for building more secure systems.

Case Study 1: Heartland Payment Systems (2008)

Heartland Payment Systems suffered a breach exposing 130 million credit card numbers. The attack began with SQL injection against the web application, leading to malware installation that intercepted unencrypted card data. Lessons: SQL injection can be the initial entry point for much larger compromises. Defense-in-depth, network segmentation, and comprehensive monitoring are critical.

Case Study 2: Sony PlayStation Network (2011)

Sony's PSN breach compromised 77 million accounts. SQL injection was suspected. The 23-day outage cost an estimated $171 million. Lessons: encrypt data at rest, implement network segmentation, maintain security patches, and invest in monitoring.

Case Study 3: TalkTalk (2015)

Teenagers exploited SQL injection in TalkTalk's website, exposing 157,000 customer records. Cost: GBP 60 million plus a record ICO fine. Lessons: legacy systems must be regularly updated, security researcher reports must be taken seriously, and sensitive data must be encrypted.

Chapter Summary

- Heartland (2008): 130M cards exposed; inadequate network segmentation.
- Sony PSN (2011): 77M accounts; cost $171M+; poor segmentation.
- TalkTalk (2015): Teenagers exploited SQLi; 157K customers; GBP 60M.
- Defense-in-depth, encryption, monitoring, and legacy updates are critical.
- Prevention is always cheaper than remediation.

Part X: Review, Challenges, and Assessments

Review Questions

1. What is the difference between DDL, DML, DCL, and TCL? Give two examples of each.

2. Explain the difference between a primary key and a foreign key.

3. What is SQL injection, and what is its root cause?

4. Describe the five types of SQL injection covered in this textbook.

5. What is a parameterized query, and why is it the most effective defense?

6. Explain the principle of least privilege in database security.

7. What is the difference between boolean-based and time-based blind SQL injection?

8. How can a WAF help prevent SQL injection? What are its limitations?

9. Describe the role of input validation in a defense-in-depth strategy.

10. What is second-order SQL injection, and why is it harder to detect?

11. How do the three security levels in the CySec Lab demonstrate progressive defense?

12. What are the ACID properties, and why are they important?

13. Describe the Docker Compose setup for the CySec Lab.

14. What lessons can be learned from the Heartland Payment Systems breach?

15. How does UNION-based SQL injection work, and what column count is required?

Practical Exercises

Exercise A: Deploy the CySec Lab using Docker Compose, complete all eight challenges at LOW security level, then repeat at MEDIUM and HIGH levels. Document the payloads that succeed at each

level and explain why they fail at higher levels.

Exercise B: Review the main.py source code in the CySec Lab. For each challenge, identify the exact line where the vulnerability exists, classify the SQLi type, and write the secure fix using parameterized queries.

Exercise C: Using phpMyAdmin, examine the login_attempts and audit_log tables after performing various injection techniques. Write a brief analysis of what monitoring data would help detect each type of attack in a production environment.

Scenario-Based Challenges

Scenario 1: You are the security lead for a mid-size e-commerce platform. A junior developer shows you a search feature using string concatenation. They argue that input validation is sufficient because the search only accepts alphanumeric characters. How do you respond? What are the risks, and what recommendations would you make?

Scenario 2: You have been hired to assess a 15-year-old banking application with direct SQL concatenation and full admin database privileges. The bank cannot afford a complete rewrite. Propose a prioritized remediation plan balancing security with practical constraints.

Scenario 3: Your monitoring system detects queries with UNION SELECT statements against your customer database. Describe the immediate incident response steps for containment, damage assessment, and prevention.

Capstone Project

Objective: Demonstrate comprehensive SQL injection defense skills using the CySec Lab. For each of the eight challenges: (1) identify the vulnerability in the source code, (2) craft a demonstration payload proving the vulnerability exists, (3) document the vulnerable versus secure code, (4) verify the fix prevents exploitation at HIGH security level. Create a comprehensive lab report with screenshots, code analysis, and reflections connecting each exercise to the textbook concepts.

Glossary

This glossary defines key terms used throughout this textbook.

ACID: Atomicity, Consistency, Isolation, Durability - the four properties that guarantee reliable database transactions.

Blind SQL Injection: SQL injection where the attacker cannot see query results directly and must infer data through behavioral differences.

CRUD: Create, Read, Update, Delete - the four basic database operations.

DCL: Data Control Language - SQL commands for managing permissions (GRANT, REVOKE).

DDL: Data Definition Language - SQL commands for defining database structure (CREATE, ALTER, DROP).

DML: Data Manipulation Language - SQL commands for manipulating data (SELECT, INSERT, UPDATE, DELETE).

Foreign Key: A column that references the primary key of another table, establishing a relationship.

In-Band SQL Injection: SQL injection where the attacker uses the same channel to inject and retrieve results.

Least Privilege: Security principle: grant only the minimum permissions necessary.

ORM: Object-Relational Mapping - a technique for working with database records as programming objects.

Out-of-Band SQL Injection: SQL injection using a secondary channel (DNS, HTTP) to exfiltrate data.

Parameterized Query: A prepared statement that separates SQL code from data, preventing injection.

Primary Key: A column (or set of columns) that uniquely identifies each row in a table.

RDBMS: Relational Database Management System - software for managing relational databases.

Rollback: A TCL command that undoes all changes made in the current transaction.

Schema: The structure of a database, including tables, columns, relationships, and constraints.

Second-Order SQL Injection: Injection where payloads are stored and later triggered in a different context.

SQL: Structured Query Language - the standard language for managing relational databases.

SQL Injection (SQLi): A code injection technique that exploits vulnerabilities in database query construction.

TCL: Transaction Control Language - SQL commands for managing transactions (COMMIT, ROLLBACK, SAVEPOINT).

Tautology: A condition that is always true, often used in SQL injection to bypass authentication.

UNION Operator: Combines the result sets of two or more SELECT statements into a single result.

WAF: Web Application Firewall - a security tool that filters and monitors HTTP traffic.

CySec Lab: The CySec Don SQLi Training Lab - custom vulnerable web application accompanying this textbook.

SQL Quick Reference Cheatsheet

Printable quick reference for SQL commands and security best practices.

SQL Command Reference

Category	Command	Description
DDL	CREATE TABLE	Create a new table
DDL	ALTER TABLE	Modify table structure
DDL	DROP TABLE	Delete table and all data
DML	SELECT ... WHERE	Query data with conditions
DML	INSERT INTO ... VALUES	Insert new records
DML	UPDATE ... SET ... WHERE	Modify existing records
DML	DELETE FROM ... WHERE	Remove records
DCL	GRANT privilege ON obj TO user	Give permissions
DCL	REVOKE privilege ON obj FROM user	Remove permissions
TCL	COMMIT	Make transaction permanent
TCL	ROLLBACK	Undo current transaction
TCL	SAVEPOINT name	Set rollback point

SQL Injection Defense Cheatsheet

Do	Do Not
Use parameterized queries for ALL SQL	Use string concatenation with user input
Validate input on the server side	Rely on client-side validation only
Apply least privilege to DB users	Use root/sa/admin as application DB user
Encrypt sensitive data at rest	Store passwords, SSNs, or cards in plain text
Log and monitor database queries	Disable database query logging in production

Keep software and patches updated	Run unpatched DBMS or framework versions
Use ORMs properly (query builders)	Use ORM raw() methods with concatenated input
Implement WAF as additional layer	Rely on WAF as the sole defense
Test in authorized lab environments	Test on systems without authorization

CySec Lab Quick Reference

Action	Command / URL
Start lab	docker compose up -d --build
View logs	docker compose logs -f app
Reset database	http://localhost:5000/reset-db
Lab dashboard	http://localhost:5000
phpMyAdmin	http://localhost:8080
Build from source	bash scripts/build.sh
Change security level	Dashboard > Security Level buttons

Secure Code Patterns

Python (mysql-connector):

```
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

PHP (PDO):

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$user_id]);
```

Java (JDBC):

```
PreparedStatement ps = conn.prepareStatement( "SELECT * FROM users WHERE id =
?"); ps.setInt(1, userId);
```

Node.js (mysql2):

```
const [rows] = await pool.execute( "SELECT * FROM users WHERE id = ?",
[userId]);
```

C# (.NET):

```
var cmd = new SqlCommand("SELECT * FROM users WHERE id = @id", conn);
cmd.Parameters.AddWithValue("@id", userId);
```